



Technische
Universität
Braunschweig

Algorithm Engineering 2026: Sheet 03

Rouven Kniep, June 04, 2026

Ex1: Setting

Mutually Exclusive Knapsack

Given: Instance with

- Items $X = \{x_1, \dots, x_n\}$
- Profits $p_i \in \mathbb{R}_{\geq 0}$
- Weights $w_i \in \mathbb{R}_{\geq 0}$
- Capacity $c \in \mathbb{R}_{\geq 0}$
- Excluded items $E_i \subseteq X$
(symmetrical)

Constraints and Target

- find solution $S \subseteq X$ with:
- $\max \sum_{i=1}^n p_i * x_i$
(optimization function), subject to:
- $\sum_{i=1}^n w_i x_i \leq c$ (within capacity)
- $\forall i : \forall j \in E_i : x_i + x_j \leq 1$
(mutually exclusive)
- $\forall i : x_i \in \mathcal{B} = \{0; 1\}$ (binary variables)

Ex1a: Constraint Propagation (overview)

Terminology

what?	best found solution	best possible value
validity	global	local (node)
min.	upper bound	lower bound
max.	lower bound	upper bound
Gurobi	incumbent	bound
CP-SAT	objective	bound

In each node:

1. dequeue node
2. if incumbent $\geq$ bound: next node
3. **constraint propagation** - extend node's partial solution
4. get bound + improve bound with cuts
5. update stats
6. if incumbent $\geq$ bound: next node
7. otherwise: generate + enqueue branches

Ex1a: Constraint Propagation

given: partial assignments $P: \text{list}[\text{bool} | \text{None}]$

constraints: mutually exclusive - w.l.o.g. let $x_i + x_j \leq 1$ and $P_i \in \text{bool}$

- if $P_i = \text{False}$: constraint unnecessary
 - if $P_i = \text{True}$: boolean case: $\implies P_j = \text{False}$; fractional case: $\implies x_j^* = 0$
 - $\implies$ no gain to be had
-

constraint: capacity $\sum_{i=1}^n w_i x_i \leq c$

- let $rem = c - \sum_{i=1}^n (w_i \mid P_i = \text{False})$
- w.l.o.g let $\exists j \mid w_j < rem$
- boolean case: $\implies P_j = \text{False}$
- fractional case: $\implies x_j^* \in [0; 1)$
- $\implies$ relaxation may assume impossible values $\implies$ propagate!

Ex1a: Constraint Propagation (solution)

```
1 def constraint_propagation(self, partial_assignment: list[bool | None]):
2     remaining_capacity = self.instance.capacity = sum(
3         self.index_to_item[i].weight
4         for i, assigned in enumerate(partial_assignment)
5         if assigned
6     )
7     for i, assigned in enumerate(partial_assignment):
8         if assigned is None:
9             if self.index_to_item[i].weight > remaining_capacity:
10                partial_assignment[i] = False
```

Ex1a: Constraint Propagation (wrong solution)

```
1 def constraint_propagation(self, partial_assignment: list[bool | None]):
2     remaining_capacity = self.instance.capacity
3     for i, assigned in enumerate(partial_assignment)
4         if assigned is None:
5             if self.index_to_item[i].weight >= remaining_capacity:
6                 partial_assignment[i] = False
7             elif assigned is True:
8                 remaining_capacity -= self.index_to_item[i].weight
```

Ex1a: Constraint Propagation (wrong solution)

```
1 def constraint_propagation(self, partial_assignment: list[bool | None]):
2     remaining_capacity = self.instance.capacity
3     for i, assigned in enumerate(partial_assignment)
4         if assigned is None:
5             if self.index_to_item[i].weight >= remaining_capacity:
6                 partial_assignment[i] = False
7             elif assigned is True:
8                 remaining_capacity -= self.index_to_item[i].weight
```

- wrong comparison (ln. 5): would disable fitting items
⇒ INCORRECT! Algorithm will throw out valid solutions!
- merged loops: remaining_capacity is decreased after propagation check
⇒ valid, but too weak: does not disable all eligible items

Ex1b: Heuristic

```
1  for rd in range(ROUNDS): # ROUNDS = 1000 - still 0.1s!
2      result = [False] * len(self.instance.items)
3      candidates = sorted_idx.copy(); excludes = set()
4      profit = 0.0; remaining_cap = self.instance.capacity
5      while candidates:
6          num_choices = min(len(candidates), NUM_CANDIDATES)
7          i = random.choice(candidates[:num_choices])
8          candidates.remove(i); item = self.item_to_index[i]
9          if item.weight <= remaining_cap and item.name not in excludes:
10             remaining_cap -= item.weight; profit += item.profit
11             result[i] = True; excludes.update(item.excludes)
12  if profit > best_profit:
13      best_profit = profit; best_result = result
```

Ex1c: Knapsack Cuts

Flow:

1. calculate relaxation
2. if relaxation is int: return
3. if cut_rounds++ > max_cut_rounds: return
4. try to find cuts
5. if no cuts found: return
6. add cuts + repeat

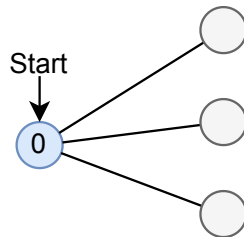
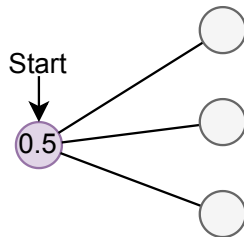
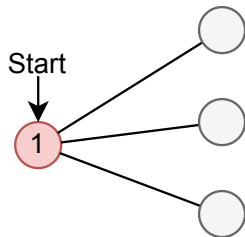
Idea: find $D \in [1; n]$, s.t.

1. $\sum_{i \in D} w_i > c$
2. $\sum x_i^* > |D| - 1$

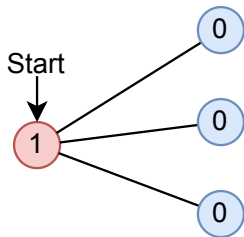
Ex1c: Knapsack Cuts

```
1 best_cut_indices = None; best_violation = 0.0
2 for sorted_indices in (sorted_indices_1, sorted_indices_2):
3     weight_sum = 0.0; value_sum = 0.0; cut_indices = []
4     for i in sorted_indices:
5         weight_sum += self.index_to_item[i].weight
6         value_sum += solution[i]; cut_indices.append(i)
7         violation = value_sum - (len(cut_indices) - 1) * CUT_VIOLATION_TOLERANCE
8         if weight_sum > capacity_with_buffer:
9             if violation > best_violation:
10                 best_violation = violation
11                 best_cut_indices = cut_indices.copy()
12             break
13     if violation <= best_violation:
14         break
```

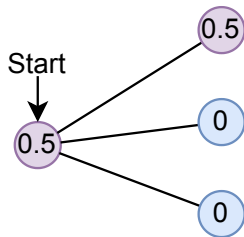
Ex1d: Clique Cuts - which items to include



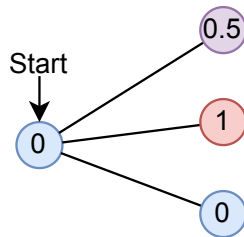
Ex1d: Clique Cuts - which items to include



all candidates have value 0
 $\Rightarrow$ no cuts

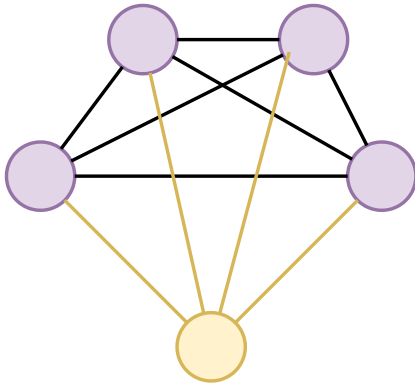


candidates have values
0-0.5 $\Rightarrow$ interesting!



unlikely (interesting cliques
found from 2.)

Ex1d: Clique Cuts - Clique Size



- avoid items of value 1
- maximum clique constraints imply all smaller ones:

$$x_1^* + x_2^* + x_3^* + x_4^* + x_5^* \leq 1$$

$$\implies x_1^* + x_2^* + x_3^* + x_4^* \leq 1$$

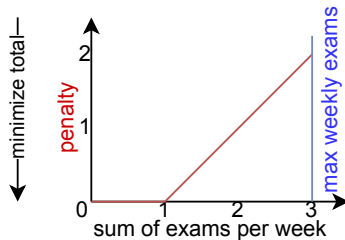
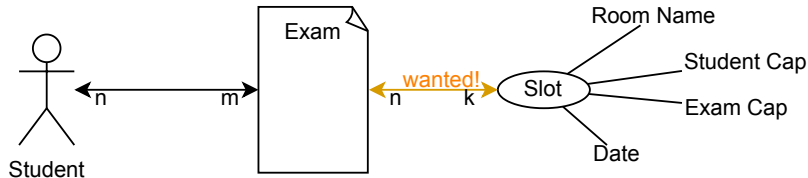
Ex1d: Clique Cuts

```
1 def clique_cut_separation(self, solution: Sequence[float]) -> list[int] | None:
2     sol = list(solution)
3     candidates = [i for i, value in enumerate(sol)
4                   if (value > INTEGRALITY_TOLERANCE
5                       and value < 1 - INTEGRALITY_TOLERANCE)]
6     candidates.sort(key=lambda i: sol[i], reverse=True)
7     best_clique: list[int] = []; best_value = 0.0
8     # inner loop, see next slide
9     if best_value <= 1.0 + CUT_VIOLATION_TOLERANCE:
10         return None
11     else:
12         return best_clique
```

Ex1d: Clique Cuts

```
1 // candidates are without 1 and 0 -> all are interesting
2 for anchor in candidates[:20]:
3     clique = [anchor]; value = solution[anchor]
4     extensions = self.exclusion_graph[anchor].copy()
5     while extensions:
6         best = min(extensions, key=lambda i: solution[i])
7         clique.append(best); value += solution[best]
8         extensions.intersection_update(self.exclusion_graph[best])
9     if value > best_value:
10         best_value = value; best_clique = clique.copy()
```

Ex2: Exam Scheduling - Setting



- max 1 exam per day foreach student
- limit of exams per slot
- all exams scheduled
- all exams scheduled
- exam students must fit slot student cap

Variables per Sitting? (wrong!)

```
1  def _make_vars(self):
2      self.vars = {
3          (exam, slot, sitting):
4              self.model.new_bool_var(
5                  name=f"({exam},{slot},{sitting})"
6              )
7          for exam in self.exams
8          for slot in self.instance.slots
9          for sitting in range(slot.max_exams)
10     }
```

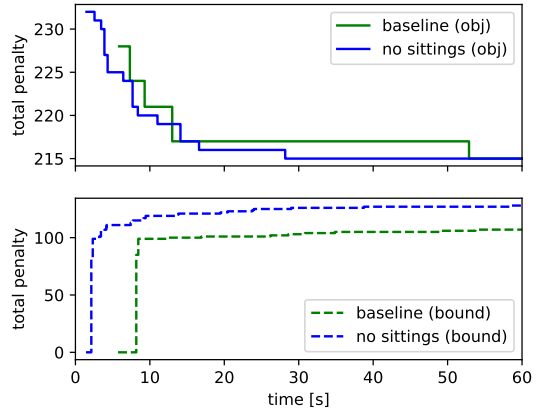
- Idea: assign each sub-slot in exams per day individually
- so: morn-
ing/noon/afternoon/evening...
- but: not necessary (we don't need the exact sub-slot for anything)

Exam-Slot-Variables (good!)

```

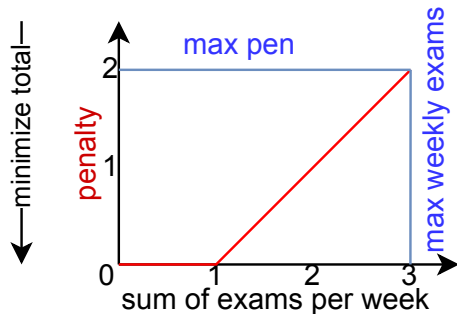
1 def _make_vars(self):
2     self.vars = {
3         (exam, slot):
4             self.model.new_bool_var(
5                 name=f"({exam},{slot})"
6             )
7         for exam in self.exams
8         for slot in self.instance.slots
9     }

```



⇒ only model needed complexity

Objective



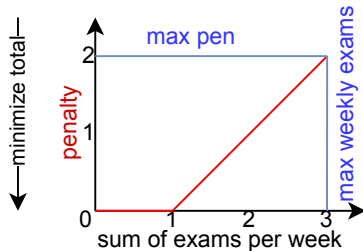
- penalty 0 if sum of exams ≤ 1 , +1 for each more
- not linear! $\Rightarrow$ additional penalty-variables needed

```

1  for student, exams in self.students.it
2      for isoweek, slots in self.isoweeks
3          penvar = self.model.new_int_var(
4              0, 3 - 1,
5              f"pen_{student}_{isoweek}")
6          self.model.add_max_equality(
7              penvar, 0, sum(
8                  self.vars[(exam, slot)]
9                  for exam in exams
10                 for slot in slots
11             ) - 1)
12         self.penalties.append(penvar)
13 self.model.minimize(sum(self.penalties

```

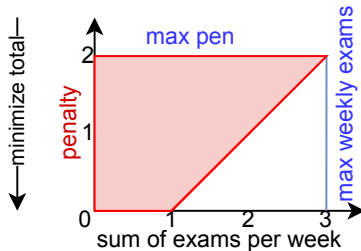
Objective (improved)



```

1 self.model.add_max_equality(
2     penvar, 0, sum(
3         self.vars[(exam, slot)]
4         for exam in exams
5         for slot in slots
6     ) - 1)

```

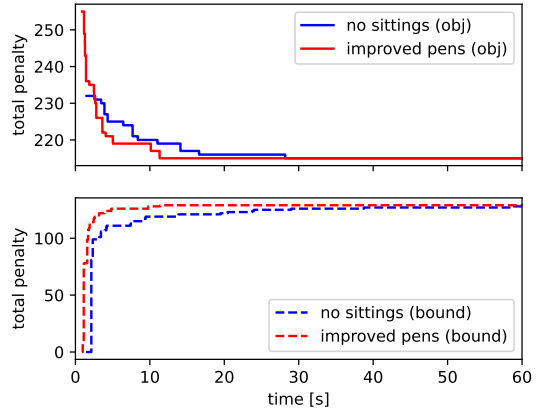
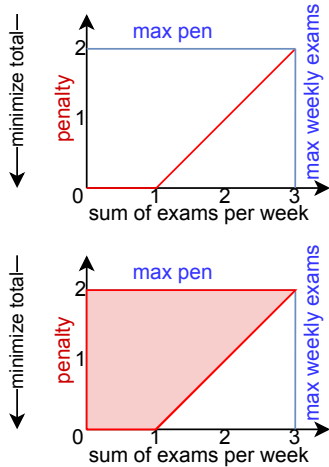


```

1 self.model.add(penvar + 1
2     >= sum(self.vars[(exam, slot)]
3         for exam in exams
4         for slot in slots)
5 )

```

Objective (improved)



⇒ avoid unnecessary non-convex constraints

Room constraints

```
1 def _exam_constraints(self):
2     # Capacity
3     for (exam, slot), var in self.vars.items():
4         self.model.add(var * self.exams[exam] <= slot.capacity)
5     # Held once
6     for exam in self.exams:
7         self.model.add_exactly_one(
8             self.vars[(exam, slot)] for slot in self.instance.slots
9         )
```

Room constraints

```
1 def _exam_constraints(self):
2     # Capacity
3     for (exam, slot), var in self.vars.items():
4         self.model.add(var * self.exams[exam] <= slot.capacity)
5     # Held once
6     for exam in self.exams:
7         self.model.add_exactly_one(
8             self.vars[(exam, slot)] for slot in self.instance.slots
9         )
```

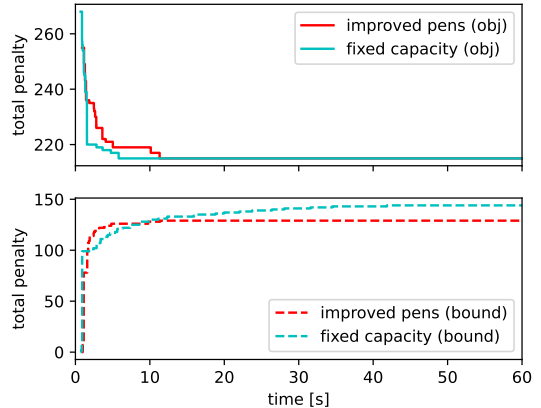
- relaxes poorly: fractional values caused by room cap
(example: 45 cap and 50 students $\implies \text{var} \leq 0.9$)
- room cap and number of students known during model creation
 $\implies$ could force variable to 0 or not even make variable

Room Constraints (improved)

```

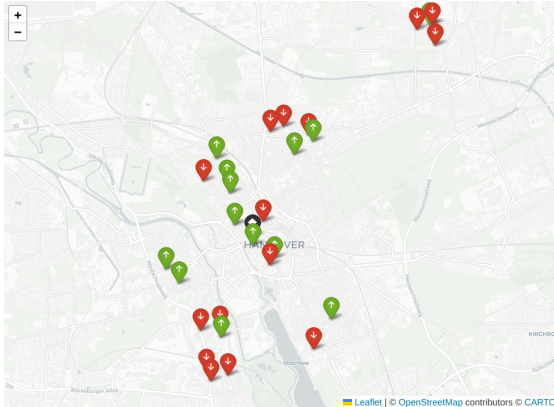
1 def _make_vars(self):
2     self.vars = {(exam, slot):
3         self.model.new_bool_var(
4             name=f"({exam},{slot})")
5         for exam, num_students
6             in self.exams.items()
7         for slot in self.instance.slots
8         if num_students <= slot.capacity
9     }
10    # and a check that all exams have
11    # at least one feasible slot

```



⇒ only model constraints and variables
about data not known during construction!

Exercise 3: Bike Redistribution



- bikes: pick up and drop off at locations
- vehicle with max capacity
- tour from and to depot
- distances: given matrix
- wanted: shortest tour

Decision Variables

```
1  def _make_vars(self):
2      # arcs: directed edge from node *u* to *v*
3      # directed for distance, also performs action at *v*
4      self.arc_vars = {
5          (u, v): self.model.new_bool_var(f"arc[{u}->{v}]")
6          for (u, v) in itertools.permutations(self.all_nodes, 2)
7      }
8      # load: loaded bikes after stop at node *n*
9      self.load_vars = {
10         n: self.model.new_int_var(0, self.instance.capacity, f"load[{n}]")
11         for n in self.all_nodes
12     }
```

Objective

```
1 def _objective(self):  
2     # min total travel distance  
3     self.model.minimize(  
4         sum(self.distances[u, v] * lit  
5         for (u, v), lit in self.arc_vars.items())  
6     )
```

- perfectly linear
- no additional helper-variables necessary
- good relaxations...

Capacity Constraints

```
1 def _capacity_link_constraints(self):
2     # Channelling: along any used arc (u, v): load[v] = load[u] + delta[v].
3     self.model.add(self.load_vars[self.depot] == 0)
4     for (u, v), lit in self.arc_vars.items():
5         if v == self.depot:
6             self.model.add(self.load_vars[u] == 0).only_enforce_if(lit)
7         else:
8             self.model.add(
9                 self.load_vars[v] == self.load_vars[u] + self.delta[v]
10            ).only_enforce_if(lit)
```

- directly based on connection $\implies$ no helpers / no complex implications

Circuit Constraints (MTZ - Bad!)

```

1  def _circular_constraint_mtz(self):
2      self.order_vars = {n: self.model.new_int_var(
3          lb=0, ub=len(self.all_nodes) - 1, name=f"order_{n}")
4          for n in self.all_nodes}
5      self.model.add(self.order_vars[self.depot] == 0)
6      self.model.add_all_different(self.order_vars.values())
7      for (u, v), lit in self.arc_vars.items():
8          target = len(self.all_nodes) - 1 if v == self.depot
9                  else self.order_vars[v] - 1
10         self.model.add(self.order_vars[u] == target).only_enforce_if(lit)
11         # + one incoming + one outgoing for every node

```

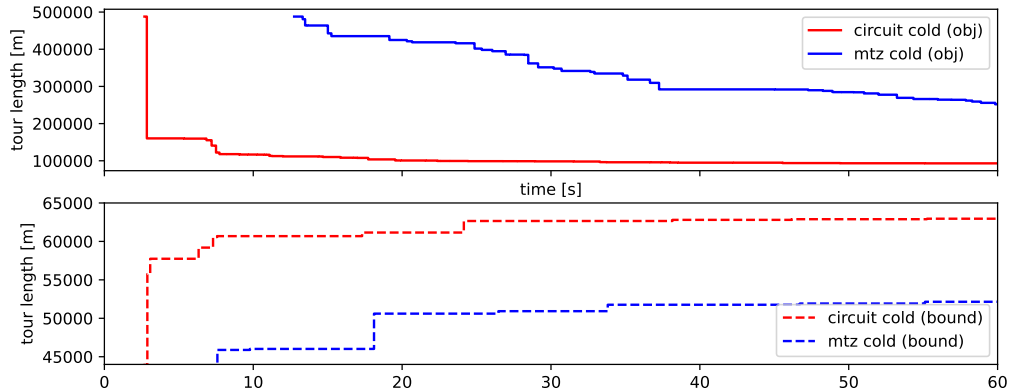
- horrible relaxations in the tour constraint!
- MTZ has a very large (relaxed) polytope

Circuit Constraints (CP-SAT add_circuit)

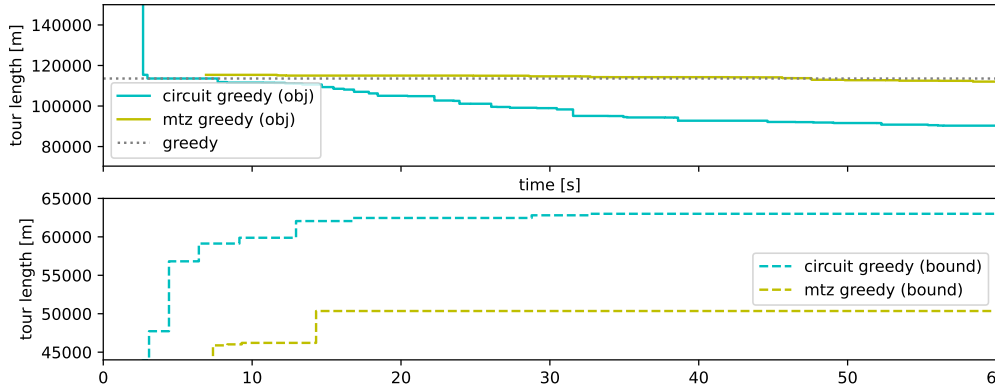
```
1 def _circular_constraint(self):  
2     self.model.add_circuit([  
3         (self.node_id[u], self.node_id[v], var)  
4         for (u, v), var in self.arc_vars.items()  
5     ])
```

- high-level abstraction
- availability and performance depends on solver...
- ... if available often VERY good!

MTZ vs add_circuit (no warm-start)

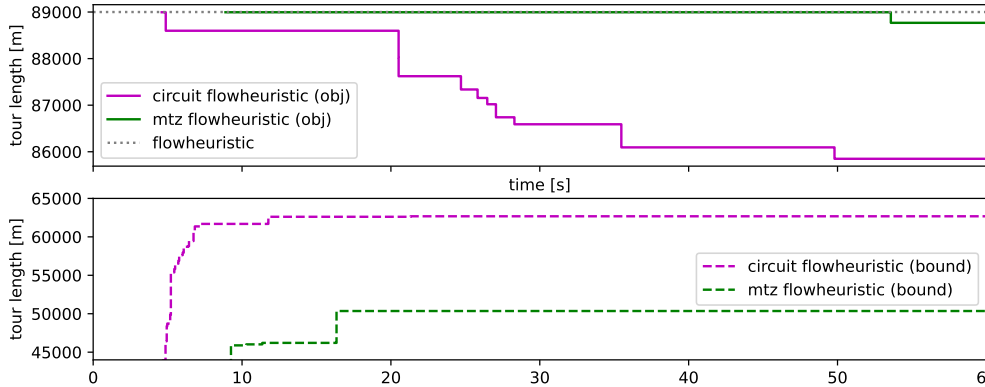


MTZ vs add_circuit (greedy warm-start)



a good heuristic can save a bad model from finding no solution, but cannot make a bad model good

MTZ vs add_circuit (specialized heuristic warm-start)



a good heuristic can save a bad model from finding no solution, but cannot make a bad model good

